

Ordenação de objetos com Bubble Sort

Comparable, Comparator e diferentes critérios de ordenação

Objetivo

Ordenando objetos, implementar e analisar a ordenação de objetos da classe Pessoa utilizando a classe genérica BubbleSort<T>.

1. Criação da classe Pessoa

Crie a classe Pessoa no mesmo pacote das classes BubbleSort e App.

```
import java.time.LocalDate;

public class Pessoa implements Comparable<Pessoa> {
    private String nome;
    private LocalDate dataNascimento;

    public Pessoa(String nome, LocalDate dataNascimento) {
        this.nome = nome;
        this.dataNascimento = dataNascimento;
    }

    public String getNome() {
        return nome;
    }

    public LocalDate getDataNascimento() {
        return dataNascimento;
    }

    @Override
    public String toString() {
        return "Pessoa{" +
            "nome=\"" + nome + "\" +
            ", dataNascimento=\"" + dataNascimento +
            "\"}";
    }

    @Override
    public int compareTo(Pessoa outra) {
        return this.nome.compareTo(outra.nome);
    }

    @Override
    public int hashCode() {
        return nome.hashCode();
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;

        if (obj == null || getClass() != obj.getClass())
            return false;

        Pessoa outra = (Pessoa) obj;
        return nome.equals(outra.nome);
    }
}
```

```
}  
}
```

Questão 1 — Análise da classe

- Para que servem a declaração `implements Comparable<Pessoa>` e o método `compareTo()` na classe `Pessoa`?

São utilizados para comparar `Pessoa` com `Pessoa` utilizando o nome como comparação.

- O que representam os valores negativo, zero e positivo retornados pelo `compareTo()`?

São possíveis valores que aparecem depois da comparação. O Java utiliza o Código Unicode para comparação onde o A começa com 65 e assim por diante.

- O que aconteceria ao chamar `sort(pessoas)` se `Pessoa` não implementasse `Comparable<Pessoa>`?
- Aconteceria erro pois Java não saberia qual característica da classe `Pessoa` comparar

2. Criação da classe App

Crie a classe `App` no mesmo pacote das classes `Pessoa` e `BubbleSort`. Execute o programa e observe a ordem dos objetos no vetor.

```
import java.time.LocalDate;  
import java.util.Arrays;  
  
public class App {  
    public static Pessoa[] vetorDePessoas() {  
        return new Pessoa[]{  
            new Pessoa("João", LocalDate.of(1990, 1, 1)),  
            new Pessoa("Ana", LocalDate.of(1992, 2, 2)),  
            new Pessoa("Maria", LocalDate.of(1995, 5, 5)),  
            new Pessoa("Pedro", LocalDate.of(1993, 4, 4)),  
            new Pessoa("Lucas", LocalDate.of(1991, 6, 6)),  
            new Pessoa("Carlos", LocalDate.of(1988, 3, 3))  
        };  
    }  
  
    public static void main(String[] args) {  
        BubbleSort<Pessoa> bubbleSortPessoas = new BubbleSort<>();  
        Pessoa[] pessoas = vetorDePessoas();  
  
        bubbleSortPessoas.sort(pessoas);  
        System.out.println(Arrays.toString(pessoas));  
    }  
}
```

Questão 2 — Ordenação natural

- Na instrução `bubbleSortPessoas.sort(pessoas)`, qual atributo é considerado na ordenação?
- Nome,
- Em qual parte da classe `Pessoa` esse critério foi definido?

```
@Override  
public int compareTo(Pessoa outra) {  
    return this.nome.compareTo(outra.nome);  
}
```

Ordenação de objetos com Bubble Sort